



Documentação Automática e Pipelines com IA

**Documentação automática: Swagger/OpenAPI, README, diagramas,
instruções de instalação**

MÓDULO 1 SEMANA 07 - DIA 01



SCITI . SC

AGENDA

- Documentação técnica assistida por IA
- Apresentação da API de Mini-Projetos
- Geração automática de README e instalação
- Documentação de arquitetura e diagramas
- Swagger/OpenAPI para documentação de endpoints
- Revisão crítica da documentação gerada
- Desafio prático guiado

OBJETIVO

Ao final da aula você será capaz de:

- Identificar os principais documentos de uma API
- Usar IA para gerar documentação técnica inicial
- Criar README, instalação e arquitetura
- Gerar diagramas técnicos com Mermaid
- Documentar endpoints com Swagger/OpenAPI
- Validar se a documentação gerada está coerente com o código

CRONOGRAMA

Semana 7

- **Segunda (18/05):** Criação de pipelines automatizados (CI/CD) utilizando sugestões da IA
- **Terça (19/05):** Documentação automática: Swagger/OpenAPI, README, diagramas, instruções de instalação
- **Quinta (20/05):** Documentação viva: atualizações contínuas guiadas por IA

Por que documentar é difícil?

1. Pressa para entregar funcionalidades

A equipe prioriza código, testes e correções urgentes.

2. Documentação feita no final

Quando fica para depois, muitos detalhes se perdem.

3. Mudanças frequentes no projeto

Endpoints, regras, telas e comandos mudam durante o desenvolvimento.

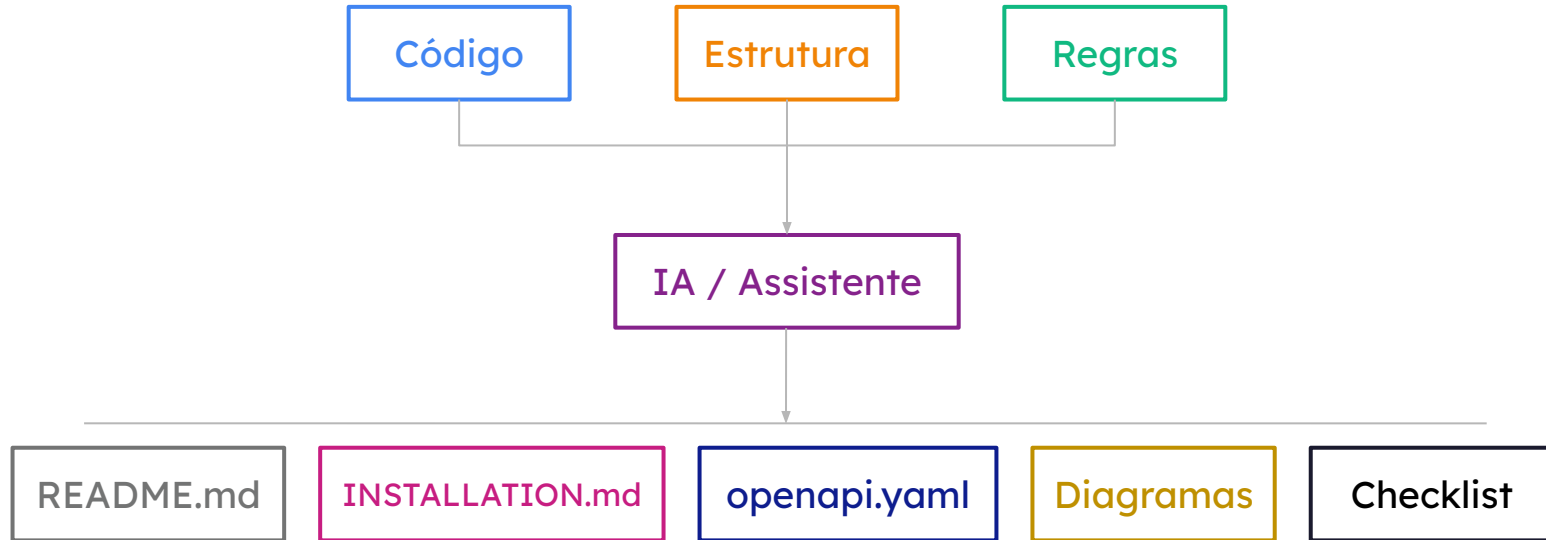
4. Falta de padrão

Cada pessoa documenta de um jeito, dificultando leitura e manutenção.

5. Baixa revisão técnica

Documentos podem conter comandos quebrados, rotas antigas ou informações inventadas.

O que é documentar automaticamente?



Artefatos técnicos de documentação

1. README.md

Visão geral do projeto, tecnologias, execução e endpoints principais.

2. INSTALLATION.md

Passo a passo para instalar, configurar e rodar o projeto.

3. openapi.yaml

Contrato técnico da API com rotas, métodos, parâmetros, bodies e respostas.

4. Diagramas Mermaid

Representação visual da arquitetura e dos fluxos da aplicação.

5. Checklist de revisão

Validação final para conferir se a documentação está coerente com o código.

Projeto base da demonstração

FeedbackHub Docs Demo

API REST para feedbacks de usuários

Projeto

Node.js + Express
Feedbacks
Dados em memória
API REST

Documentação gerada

README.md
INSTALLATION.md
openapi.yaml
Mermaid

README como visão do projeto

Seção	Pergunta respondida
Propósito	O que o projeto faz?
Stack	Quais tecnologias usa?
Execução	Como rodar localmente?
Recursos	O que existe no projeto?
Observações	Quais pontos exigem atenção?

Prompt para gerar README

Seção
Contexto
Stack
Funcionalidades
Estrutura
Restrições

O que informar?
Nome e objetivo
Tecnologias utilizadas
Funcionalidades reais
Seções esperadas
O que evitar

Prompt para gerar README

Analise o projeto abaixo e gere um README.md profissional, claro e objetivo.

Contexto:

O projeto se chama FeedbackHub Docs Demo. Ele é uma API REST simples feita com Node.js e Express.

A API permite listar feedbacks, buscar feedback por ID, cadastrar feedback, alterar status e consultar métricas.

Estrutura esperada do README:

1. Nome do projeto
2. Descrição
3. Tecnologias utilizadas
4. Funcionalidades
5. Estrutura de pastas

6. Como instalar
7. Como executar
8. Endpoints principais
9. Exemplos de requisição
10. Observações finais

Regras:

- Use Markdown.
- Não invente funcionalidades que não existem no código.
- Não mencione banco de dados, autenticação ou frontend.
- Use uma linguagem didática para alunos iniciantes.
- Gere apenas o conteúdo do README.md.

Instalação e execução local

01

Pré-requisitos

Node.js, npm e Git

02

Clone

Baixar o repositório

03

Dependências

Como instalar os pacotes

04

Execução

Iniciar a aplicação

05

Validação

Confirmar se a API
responde

06

Problemas comuns

Erros esperados e
soluções

Prompt para gerar INSTALLATION

Com base neste projeto Node.js + Express, gere um arquivo docs/INSTALLATION.md.

O documento deve explicar como uma pessoa deve preparar o ambiente e executar a API localmente.

Inclua:

1. Pré-requisitos
2. Como clonar o repositório
3. Como acessar a pasta do projeto
4. Como instalar dependências
5. Como executar em modo de desenvolvimento
6. Como executar em modo padrão
7. Como validar se a API está funcionando
8. Problemas comuns e possíveis soluções

Regras:

- Use Markdown.
- Use comandos em blocos de código.
- Não invente Docker, banco de dados ou autenticação.
- Considere que o projeto roda na porta 5000.
- Gere apenas o conteúdo do arquivo docs/INSTALLATION.md.

OpenAPI como contrato da API

Exemplo simplificado de contrato

ROTAS

Endpoints disponíveis

MÉTODOS

Operações HTTP

PARÂMETROS

Dados de entrada

BODY

Corpo da requisição

RESPOSTAS

Status code e retorno

SCHEMAS

Estrutura dos dados

GET /feedbacks/{id}

parameters:
id: number

responses:
200: Feedback encontrado
404: Feedback não encontrado

schema:
Feedback:
id
author
message
status

POST /feedbacks

body:
author
message

responses:
201: Feedback criado
400: Campos obrigatórios ausentes

schema:
Feedback:
id
author
message
status

Prompt para gerar OpenAPI

01

[Rotas reais]

GET /health
GET /feedbacks
POST /feedbacks

02

[Regras da API]

Feedback possui
id, author,
message e status

03

[Entradas]

POST /feedbacks
exige author e
message

04

[Respostas]

201: criado
400: erro de validação
404: não encontrado

05

[Restrições]

Não inventar
autenticação,
banco de dados
ou endpoints
extras

Prompt para gerar OpenAPI

Analise o backend do projeto FeedbackHub CI e gere ou corrija o arquivo openapi.yaml.

Objetivo:

Criar uma especificação OpenAPI 3.0 coerente com o comportamento real da API.

Tarefas:

1. Identifique automaticamente as rotas Express existentes.
2. Verifique métodos HTTP, parâmetros, request body e responses reais.
3. Confira os campos obrigatórios usados no backend.
4. Identifique valores válidos para campos como category, status e priority.
5. Gere ou atualize o openapi.yaml com info, servers, paths e components.
6. Atualize os exemplos de request e response para refletirem os campos e retornos reais do backend.

7. Remova campos, status codes ou endpoints que não existam no código.

Regras:

- Use OpenAPI 3.0.3 em YAML.
- Use o server `http://localhost:5000`.
- Não invente banco de dados, autenticação, login ou endpoints extras.
- A documentação deve refletir o código real.
- Gere apenas o conteúdo final do openapi.yaml.

Validação no Swagger Editor



1. Carregar o openapi.yaml

Inserir o conteúdo gerado pela IA no Swagger Editor.

2. Verificar erros de sintaxe

Conferir se o YAML é válido.

3. Conferir rotas (endpoints) e métodos

Validar se as rotas existem no código.

4. Revisar entradas e respostas

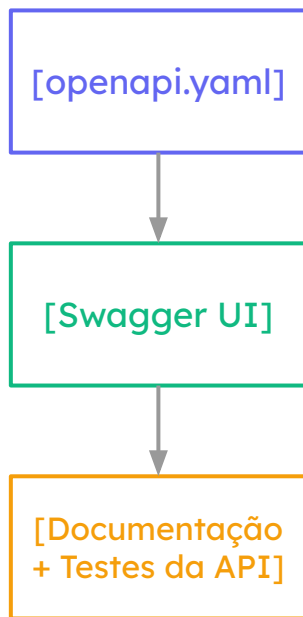
Checar parâmetros, body, status codes e schemas.

5. Ajustar inconsistências

Corrigir o arquivo antes de versionar no repositório.

Swagger UI: documentação interativa da API

O Swagger UI transforma o contrato OpenAPI em uma documentação visual, navegável e testável da API.



1. Visualizar rotas e métodos HTTP
2. Conferir parâmetros, bodies e status codes
3. Validar exemplos de entrada e saída
4. Testar endpoints diretamente pelo navegador
5. Comparar a documentação com o comportamento real do backend

No projeto, o Swagger será exposto localmente em:

<http://localhost:5000/api-docs>

Contrato da API = rotas + parâmetros + request body + responses + schemas

Prompt para adaptar o Swagger UI

Integre Swagger UI ao backend Node.js/Express existente.

Contexto:

- A API roda em `process.env.PORT` ou, por padrão, na porta 5000.
- Já existe um arquivo `openapi.yaml` no projeto.
- A documentação deve abrir em:
`http://localhost:5000/api-docs`

Tarefas:

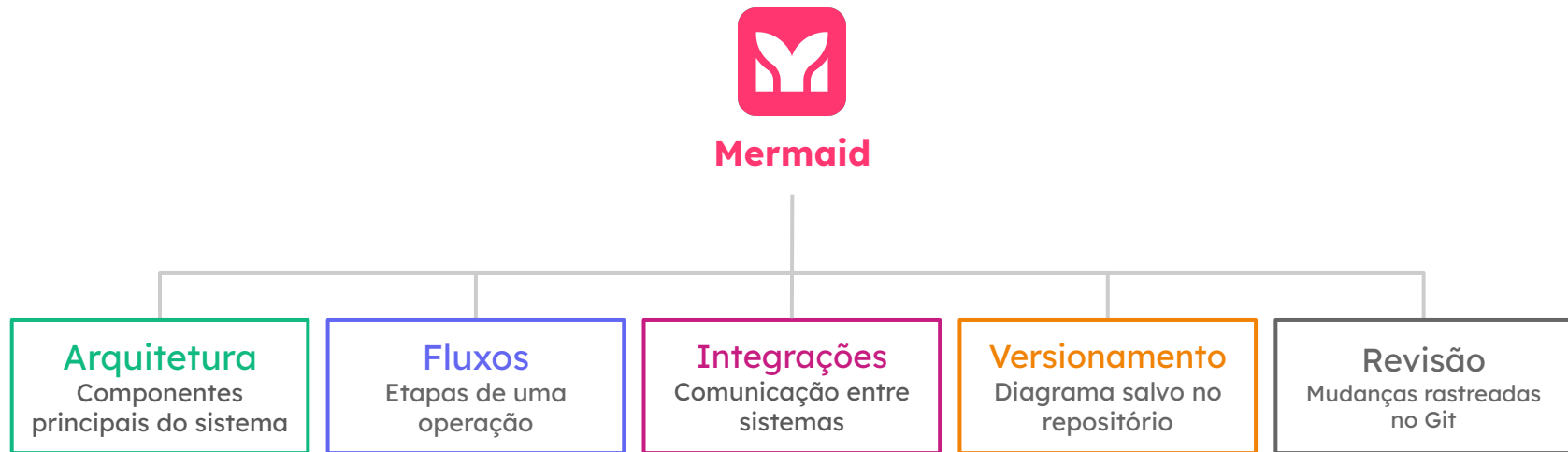
1. Localize `app.js`, `server.js` e `openapi.yaml`.
2. Adicione as dependências `swagger-ui-express` e `yamljs`.
3. Configure o app Express para carregar o `openapi.yaml` com `path.join(__dirname, ...)`.
4. Exponha a documentação na rota `/api-docs`.
5. Ajuste o `openapi.yaml` para usar o server `http://localhost:5000`.
6. Preserve todas as rotas existentes e não altere a lógica da API.
7. Mantenha o `server.js` apenas iniciando o servidor com `process.env.PORT || 5000`.

Ao final, informe:

- arquivos alterados;
- dependências adicionadas;
- comando para executar;
- URL para acessar o Swagger.

Diagramas técnicos com Mermaid

Mermaid permite criar diagramas técnicos em texto, facilitando revisão e versionamento.



Prompt para diagramas Mermaid

A IA precisa receber o tipo de diagrama, os componentes e o nível de detalhe esperado.

01

[Tipo de diagrama]

Arquitetura ou
fluxo

02

[Componentes]

Cliente, API,
rotas e dados

03

[Relações]

Comunicação
entre
componentes

04

[Escopo]

O que deve
aparecer no
diagrama

05

[Restrições]

O que não deve
ser inventado

Prompt para gerar diagrama Mermaid

Gere um diagrama Mermaid para representar o fluxo da aplicação FeedbackHub CI.

Contexto:

- O projeto possui um frontend em React.
- O frontend permite visualizar, cadastrar e atualizar feedbacks.
- O backend é uma API Node.js com Express.
- A API possui rotas para health check, feedbacks e métricas.
- Os dados são armazenados em memória.
- Não existe banco de dados, autenticação ou serviço externo.

Requisitos:

- Gere o diagrama em sintaxe Mermaid.
- Use flowchart TD.
- Mostre Usuário, Frontend React, API Express, Rotas e Dados em memória.
- Mostre o fluxo de requisição e resposta entre frontend e backend.
- Não invente banco de dados, autenticação ou serviços externos.
- Gere apenas o código Mermaid.

Prompt para gerar diagramas de caso de uso com Mermaid

Prompt – Diagrama de caso de uso

Gere o arquivo docs/diagrams/use-case.md com um diagrama de caso de uso em Mermaid.

Contexto:

O projeto FeedbackHub CI permite que o usuário gerencie feedbacks pela interface da aplicação.

Requisitos:

- Use Markdown + Mermaid com flowchart LR.
- Crie apenas o ator: Usuário.
- Represente o sistema como um subgraph chamado FeedbackHub CI.
- Inclua os casos de uso:
 - Visualizar feedbacks, Cadastrar feedback, Filtrar por status, Atualizar status, Arquivar feedback, Visualizar respostas, Cadastrar resposta e Visualizar métricas.
- Conecte o Usuário a todos os casos de uso.
- Não inclua React, Express, rotas HTTP, banco de dados, autenticação, serviços externos ou detalhes técnicos.
- Gere apenas o conteúdo final do arquivo.

Prompt para gerar diagramas de sequência com Mermaid

Prompt — Diagrama de sequência

Gere docs/diagrams/sequence-diagram.md com um diagrama Mermaid sequenceDiagram.

Contexto:

No FeedbackHub CI, o usuário cadastra feedback pela interface React. O frontend envia POST /feedbacks para a API Express. A API valida author e message, salva em memória e retorna JSON.

Requisitos:

- Use Markdown + Mermaid sequenceDiagram.
- Participantes: Usuário, Frontend React, API Express e Dados em memória.
- Mostre o fluxo de cadastro de feedback.
- Inclua sucesso: feedback salvo e resposta 201.
- Inclua erro: campo obrigatório ausente e resposta 400.
- Não inclua banco, autenticação, login ou serviços externos.
- Gere apenas o conteúdo final do arquivo.

Organização da documentação no repositório

```
feedbackhub-docs-demo/  
├── src/  
│   ├── app.js  
│   └── server.js  
├── docs/  
│   ├── INSTALLATION.md  
│   └── diagrams/  
│       └── architecture.md  
├── openapi.yaml  
├── README.md  
├── package.json  
└── .gitignore
```

1. README.md

Ponto de entrada do projeto.

2. openapi.yaml

Contrato da API em formato OpenAPI.

3. docs/INSTALLATION.md

Passo a passo para instalar e executar o projeto.

4. docs/diagrams/

Agrupar os diagramas técnicos do projeto.

5. src/

Contém o código-fonte usado como base para a documentação.

Atividade prática de documentação

Use IA para gerar os artefatos técnicos e revise tudo com base no código real.

ARTEFATO	FINALIDADE
 README.md	Visão geral, tecnologias, execução e endpoints principais.
 INSTALLATION.md	Orientar instalação e execução local do projeto.
 openapi.yaml	Formalizar o contrato da API com rotas, métodos, entradas e respostas.
 Mermaid	Representar arquitetura ou fluxos da aplicação.
 Checklist	Registrar validações e correções após a geração com IA.

Atividade prática: documentação em branch

Use IA para documentar um novo projeto e versionar os artefatos em uma branch separada.

Etapa	O que fazer?
1. Clonar	Baixar o repositório bookshelf-api
2. Criar branch	Criar a branch docs/documentacao-inicial
3. Analisar código	Identificar rotas, regras, comandos e estrutura
4. Gerar documentação	Criar README, INSTALLATION, OpenAPI e Mermaid
5. Revisar e versionar	Corrigir, commitar e enviar a branch



Documentação Automática e Pipelines com IA

Documentação viva: atualizações contínuas guiadas por IA

MÓDULO 1 SEMANA 07 - DIA 02



SCITI . SC

AGENDA

- O que é documentação viva
- Por que a documentação fica desatualizada
- Como identificar impactos de mudanças no código
- Atualização contínua de README, OpenAPI e diagramas
- Uso da IA para revisar inconsistências técnicas
- Documentação viva em branches e Pull Requests
- Checklist de revisão antes do merge
- Atividade prática

OBJETIVO

Ao final da aula você será capaz de:

- Compreender o que é documentação viva
- Identificar documentação desatualizada
- Mapear impactos de mudanças no código
- Usar IA para atualizar documentos técnicos
- Revisar README, OpenAPI e diagramas com base no projeto real
- Aplicar checklist de documentação em Pull Requests
- Manter documentação versionada junto com o código

Do documento estático à documentação viva

Documento estático

Criado uma vez e esquecido

Depende de revisão manual eventual

Pode divergir da aplicação

Descreve como o projeto era

Normalmente fica fora do ciclo de entrega

Documentação viva

Atualizada junto com o código

Entra no fluxo de branch, commit e PR

Reflete o comportamento atual do sistema

Reflete como o projeto funciona agora

Faz parte da qualidade da entrega

Desalinhamento entre código e documentação

Código evolui

- + Nova rota
- + Nova regra
- + Novo script
- + Novo fluxo
- + Nova validação

**Código
atualizado**

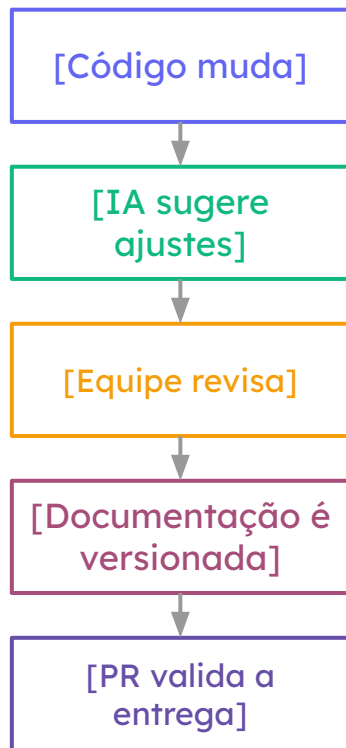
≠

**Documentação
atualizada**

Documentação estática

Endpoint ausente
Regra antiga
Comando quebrado
Diagrama incorreto
Instrução incompleta

Ciclo de manutenção da documentação viva



Resultado esperado

- Documentação alinhada ao código
- Mudanças rastreáveis no Git
- Redução de inconsistências
- Revisão técnica antes do merge
- Mais confiança na entrega

Sinais de documentação desatualizada

1 Endpoint inexistente
README diverge da API

README.md

Endpoint

GET /v1/usuarios

↓

404 Not Found !

Este endpoint não existe na API.

2 Comando quebrado
Instalação falha

Terminal

```
$ npm install
```

```
ERR! code E404
```

```
ERR! 404 Not Found -
```

```
package@2.0.0
```

Erro ao instalar dependências.

 !

3 OpenAPI incompleto
Contrato incompleto

openapi.yaml

```
paths:
  /v1/usuarios:
    get:
      responses:
        '200':
          description: OK

/v1/pedidos:
  ...
```

! Diversos endpoints e respostas ausentes.

4 Diagrama antigo
Fluxo incorreto

fluxo.png

```
graph TD
  Cliente --> API
  API --> Serviço
  Serviço --> BancoDeDados[Banco de Dados]
```

! Diagrama desatualizado: o fluxo atual não corresponde mais à arquitetura.

5 Exemplo incorreto
Resposta diferente da real

exemplos.md

Resposta esperada

```
{
  "id": 123,
  "nome": "João"
}
```

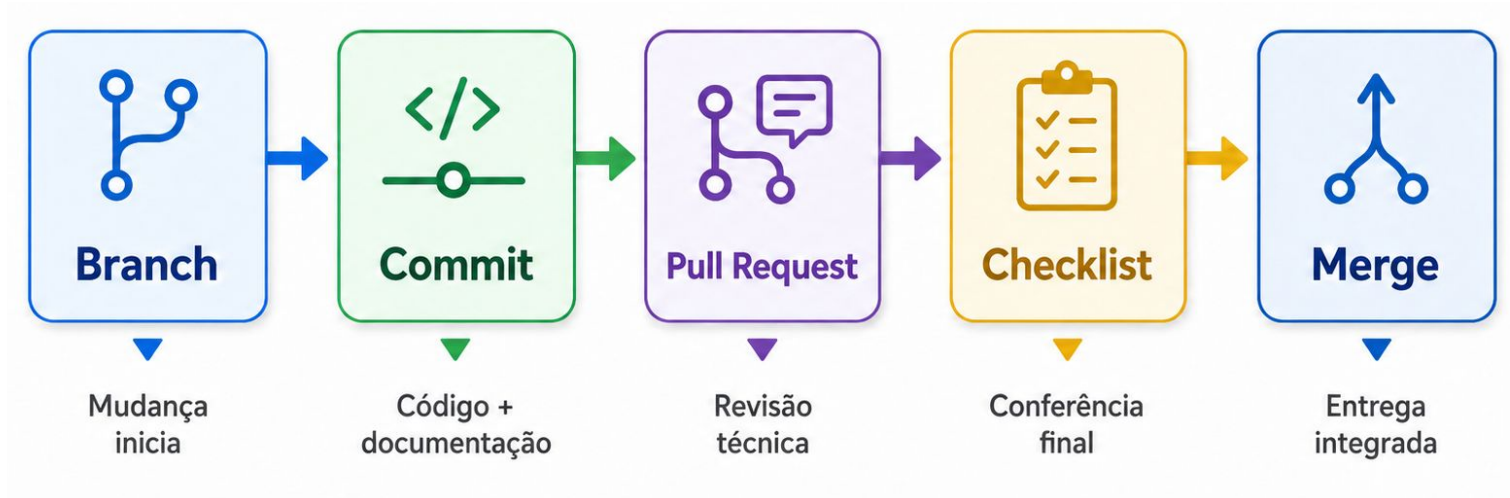
Resposta real

```
{
  "usuario_id": 123,
  "nome": "João",
  "status": "ativo"
}
```

! Exemplo não corresponde à resposta atual da API.

Quando a documentação deixa de refletir o sistema, ela passa a gerar retrabalho.

Documentação no ciclo de desenvolvimento



Mapa de impacto da documentação



Exemplo prático: nova rota na API



Exemplo prático: nova regra de negócio

Código/regra alterada → Impacto técnico → Documentos a revisar



Regra alterada

Nova regra:

Feedbacks arquivados não podem ter o status alterado diretamente.



Impacto técnico

Validação de negócio

A API deve bloquear alterações em feedbacks com status archived.



Documentos a revisar

- ✓ README.md
- ✓ openapi.yaml
- ✓ exemplos de erro
- ✓ checklist de PR

Da mudança no código à documentação viva



Mesmo processo, diferentes mudanças: [nova rota](#) | [nova regra de negócio](#) | [novo script](#) | [nova validação](#)

Desafio prático: BookShelf API



Pergunta central: Quais documentos foram realmente impactados por essa mudança?



Documentação Automática e Pipelines com IA

Criação de pipelines automatizados (CI/CD) utilizando sugestões da IA

MÓDULO 1 SEMANA 07 - DIA 03



asCTI.SC

AGENDA

- Conceito de CI/CD em projetos reais
- Fluxo com branch e Pull Request
- Validações automáticas do projeto
- Backend, frontend e documentação no pipeline
- GitHub Actions com apoio da IA
- Demonstração prática guiada

OBJETIVO

Ao final da aula você será capaz de:

- Entender a função de um pipeline CI/CD
- Diferenciar CI de CD no GitHub Actions
- Identificar validações automáticas do projeto
- Compreender o papel do pipeline na qualidade da entrega
- Usar IA para gerar e analisar workflows
- Interpretar falhas no pipeline e corrigi-las

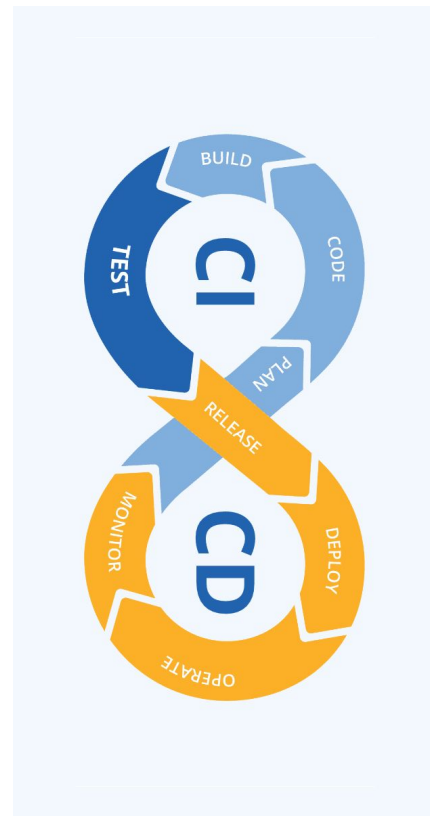
O QUE É CI/CD?

CI — Integração Contínua

- Valida mudanças antes do merge
- Executa verificações automáticas
- Ajuda a detectar erros mais cedo

CD — Entrega ou Deploy Contínuo

- Executa após as validações
- Prepara a aplicação para entrega
- Pode ser real ou simulado



POR QUE USAR PIPELINES CI/CD?

Valida

Código, testes, build e documentação

Bloqueia

Erros antes do merge

Libera

Mudanças prontas para integração

Branch → Pull Request → Pipeline → Merge

COMO UMA MUDANÇA ENTRA NO PROJETO?



Branch

Isola a
mudança

Código

Implementa
a alteração

**Pull
Request**

Propõe
integração

Checks

Executam
validações

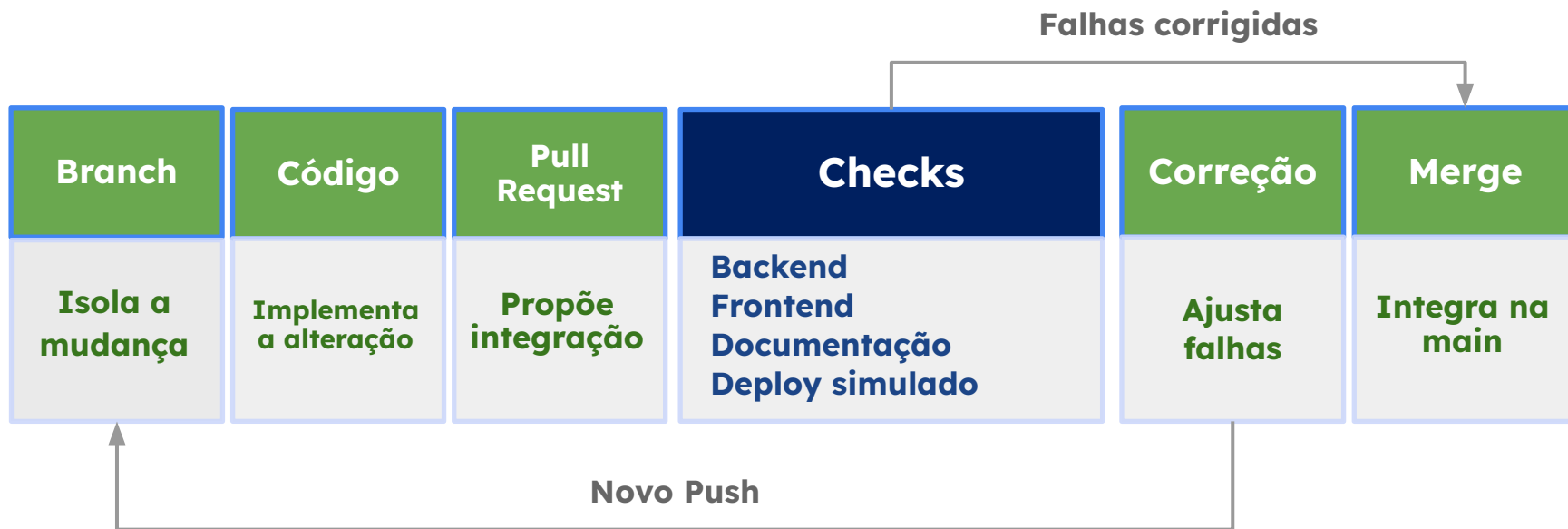
Correção

Ajusta falhas

Merge

Integra na
main

ONDE O PIPELINE ATUA?



O QUE O PIPELINE VALIDA?

01

Backend

API e regras

02

Frontend

Interface visual

03

Testes Automatizados

Comportamento
esperado

04

Build

Empacotamento

05

Documentação

README, OpenAPI e docs

06

Deploy simulado

Entrega controlada

CI E CD NO NOSSO PIPELINE

CI - Validação antes da integração

Backend

Frontend

Testes

Build

Documentação

CD — Entrega após validações

Deploy da aplicação

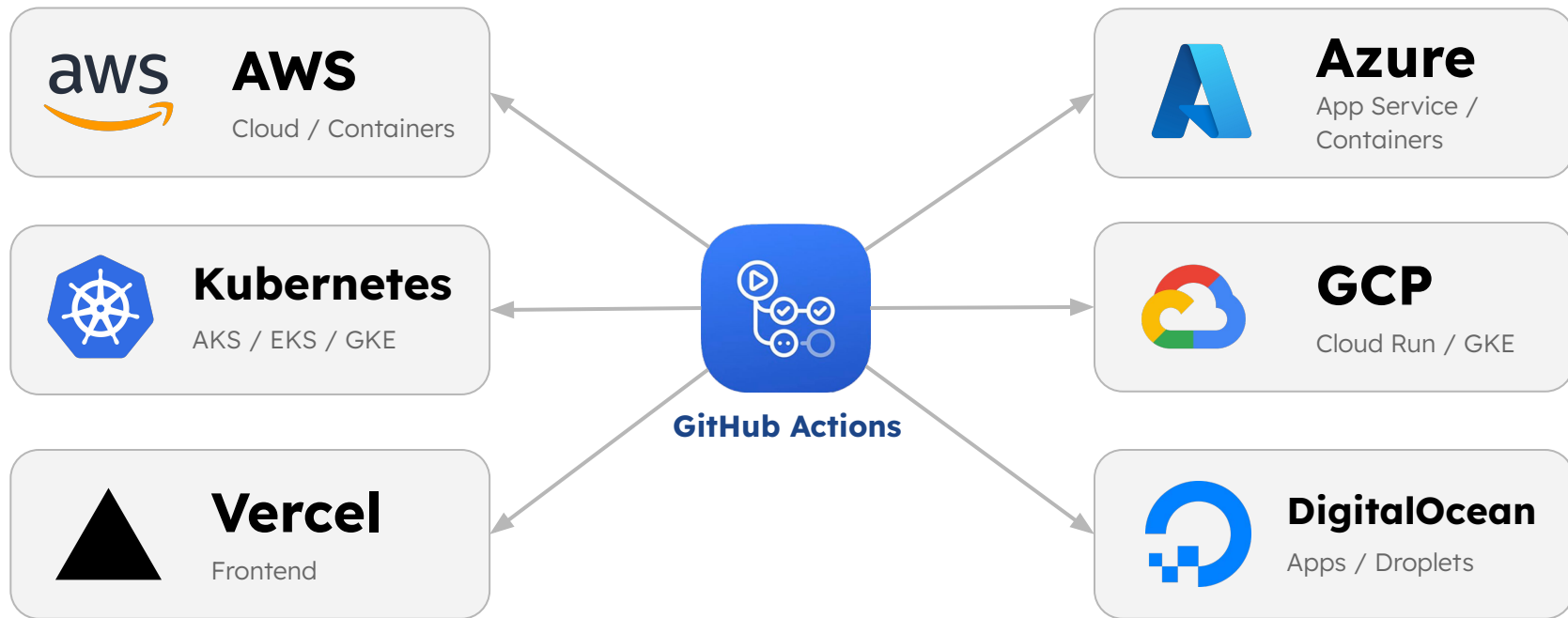
Ambiente configurado

Credenciais seguras

Status da entrega

Execução após validações

PARA ONDE O DEPLOY PODE IR?



ONDE A IA ENTRA NO PIPELINE?



1. Criar

Gera o YAML inicial

2. Explicar

Analisa erros do pipeline

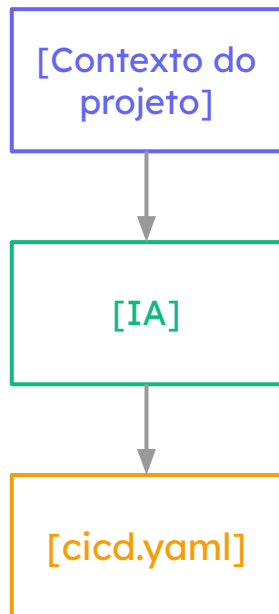
3. Melhorar

Sugere novas validações

FERRAMENTAS USADAS NO PIPELINE

Ferramenta	Onde atua	Papel no pipeline
 Lint / ESLint	Backend e frontend	Verificar qualidade e padrão do código
 Express	Backend	Implementa a API HTTP
 Jest	Backend	Executar testes automatizados
 Supertest	Backend	Testa endpoints HTTP da API
 Vite	Frontend	Gera o build da aplicação React

PROMPT PARA GERAR O WORKFLOW



Prompt

Gere um workflow GitHub Actions para uma aplicação full-stack.

Stack:

- Backend: Node.js + Express
- Frontend: React + Vite
- Testes: Jest e Vitest
- Documentação: README, OpenAPI e docs/

O pipeline deve rodar em push e pull_request. Ele deve validar backend, frontend, testes, build e documentação, e executar deploy após as validações.

PROMPT PARA GERAR O WORKFLOW

Gere um workflow GitHub Actions para uma aplicação full-stack.

Estrutura:

- backend/
- frontend/
- docs/
- README.md

Backend:

- Node.js + Express
- npm run lint
- npm test
- npm run build
- backend/openapi.yaml

Frontend:

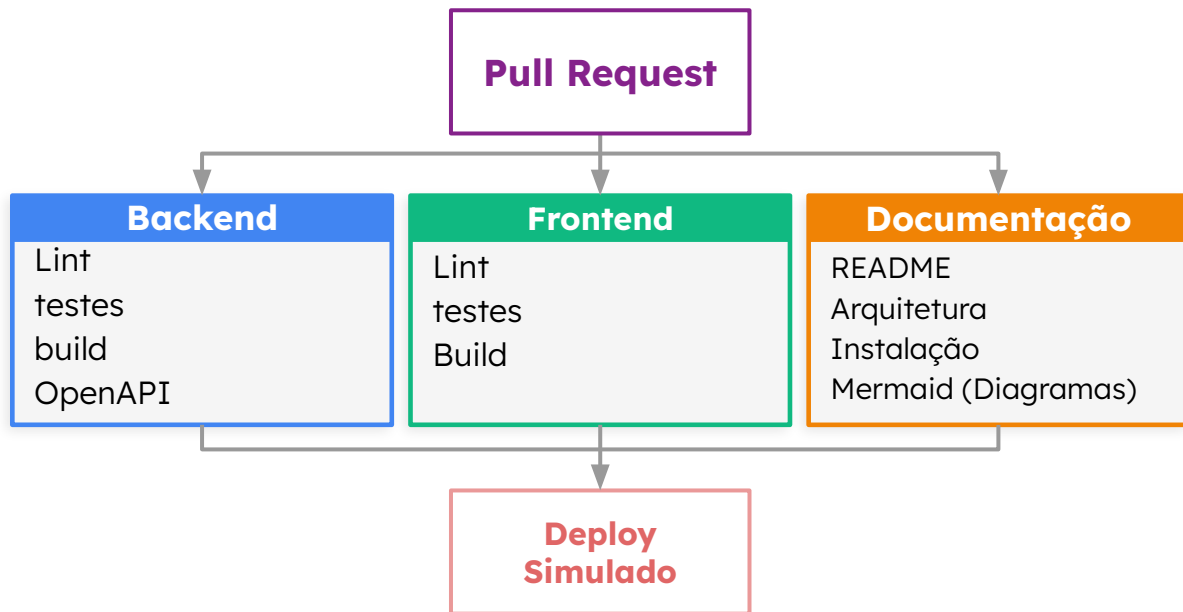
- React + Vite
- npm run lint
- npm test
- npm run build

Requisitos:

- rodar em push e pull_request para main;
- usar Node.js 20;
- criar jobs para backend, frontend, docs e deploy;
- deploy só roda se os demais jobs passarem;
- gerar apenas o YAML final.

DEMONSTRAÇÃO PRÁTICA: PIPELINE CI/CD

Vamos executar o pipeline do FeedbackHub CI e analisar os jobs de validação automática.



Só existe deploy se todos os checks passarem.

COMO REVISAR O WORKFLOW GERADO?

1. Caminhos

backend/ frontend/ docs/

2. Comandos

npm test, npm run build

3. Eventos

push, pull request

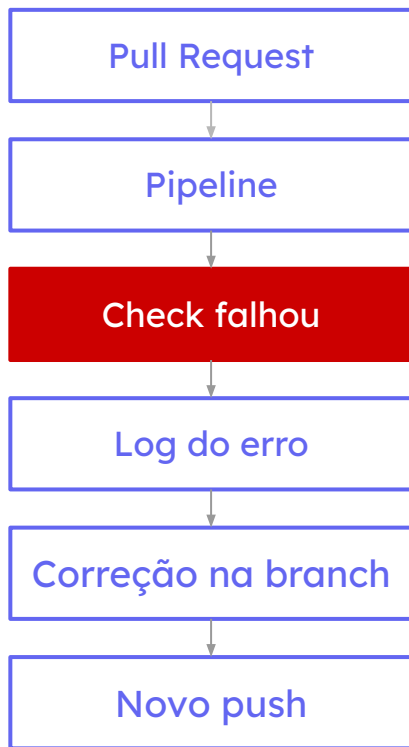
4. Segurança

secrets e variáveis de ambiente

5. Falhas

deve falhar em erros relevantes

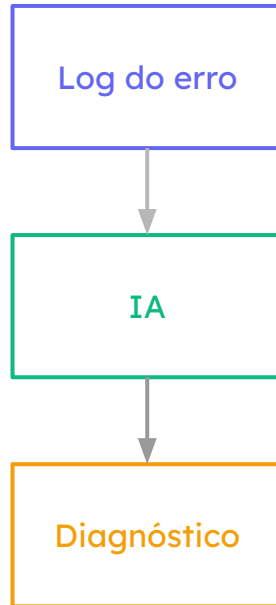
COMO REVISAR O WORKFLOW GERADO?



O QUE FAZER QUANDO FALHA?

- 1.** Ler o log
- 2.** Localizar o job
- 3.** Entender o comando
- 4.** Corrigir a causa
- 5.** Enviar novo push
- 6.** Validar novamente

IA PARA ANALISAR FALHAS NO PIPELINE



O DIAGNÓSTICO DEVE INDICAR

Job com erro

Etapa que falhou

Comando executado

Causa provável

Correção sugerida

PIPELINE CI/CD: VISÃO COMPLETA DO FLUXO

PIPELINE CI/CD GERADO POR IA — DO PUSH AO DEPLOY



YAML GERADO POR IA (GitHub Actions)

```
name: CI/CD Pipeline
on: [push, pull_request]
jobs:
  build-test-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: npm ci
      - run: npm test -- --coverage
      - run: npm run lint
      - uses: vercel/action@v1
        with: { token: ${ secrets.VERCEL }}
```

BENEFICIOS DO CI/CD COM IA

- Zero configuracao manual
- Testes rodando a cada push
- Deploy automatico sem estresse
- Cache configurado para velocidade
- Segredos gerenciados via Secrets
- Notificacao de falha no Slack
- YAML comentado e documentado
- IA atualiza o pipeline conforme a stack

EXPERIMENTOS CONTROLADOS DE FALHA

01

Teste falhando

Status esperado incorreto

02

Build quebrado

Import ou componente
inexistente

03

Documentação ausente

Arquivo obrigatório
removido

Falha → Diagnóstico → Correção → Novo push → Checks verdes

DO CHECK VERDE AO MERGE



Checks verdes não substituem revisão humana

Atividade prática: BookShelf API

Etapa	O que fazer
1. Clonar	Baixar o repositório bookshelf-api
2. Criar branch	Ambiente configurado
3. Rodar testes locais	Backend com Jest e Frontend com Vitest
4. Criar pipeline	<code>.github/workflows/ci.yml</code>
5. Revisar e versionar	Commit, push e Pull Request

O QUE LEVAR PARA A PRÁTICA?

1. Processo

Branch → PR → Checks → Merge

2. Qualidade

Testes, build e documentação

3. Entrega

Deploy em ambiente configurado

CI/CD não é só automação: é controle de qualidade no fluxo de entrega

Conclusão - Documentação Automática e Pipelines com IA

Dia 1 — Documentação automática com IA

- Geração de README, instalação, OpenAPI e diagramas
- Prompts estruturados com contexto, regras e restrições
- Revisão da documentação com base no código real

Dia 2 — Documentação viva com IA

- Diferença entre documentação estática e viva
- Análise de impacto a partir de mudanças no código
- Atualização de README, OpenAPI e diagramas afetados

Dia 3 — Pipelines CI/CD com IA

- Conceitos de CI/CD, branches, PRs e checks
- Workflows com GitHub Actions para validar o projeto
- Análise de falhas e correções com apoio da IA



SCTEC

PREPARANDO
VOCÊ PARA O
AGORA_



SCITI.SC